

Administración de recursos compartidos, sincronización, comunicación entre procesos

Sincronización: su definición y otras características

La sincronización o concurrencia está relacionada con la gestión de procesos y hebras, tanto sea en la multiprogramación como en el multiprocesamiento o en el procesamiento distribuido y se la puede definir como la propiedad de los sistemas operativos de permitir que múltiples procesos sean ejecutados al mismo tiempo, y que potencialmente puedan interactuar entre sí.

La concurrencia como propiedad de los sistemas operativos presenta las siguientes características:

- Concurrencia real: ocurre cuando, en un sistema de multiprocesamiento, hay un proceso para ser ejecutado por cada procesador permitiendo que los procesos concurrentes puedan ser ejecutados realmente de forma simultánea.
- Concurrencia simulada: presente cuando sólo existe un procesador encargado de ejecutar los procesos concurrentes, simulando la concurrencia, ocupándose de forma alternada en uno y otro proceso a pequeñísimos intervalos de tiempo, o cuando el número de procesos es mayor que el de procesadores disponibles, es decir, cuando haya más de un proceso por procesador.
- Las hebras comparten los espacios de direcciones lógicas (código y datos) y ejecutan procesos concurrentes lo que facilita su programación, permite acelerar los cálculos, posibilita el uso interactivo a múltiples usuarios y consigue un mejor aprovechamiento de los recursos de la computadora.
- Los procesos concurrentes que se ejecutan en un sistema pueden ser:
 - Independientes: son aquellos que se ejecutan sin requerir la ayuda o cooperación de otros procesos.
 - Cooperativos: son procesos que han sido diseñados para trabajar conjuntamente en alguna actividad, por lo tanto, necesitan mecanismos de comunicación y sincronización. Estos procesos pueden tener acceso concurrente a datos compartidos, que puede generar inconsistencia de datos.
- La concurrencia de accesos puede presentarse en tres contextos diferentes:
 - Múltiples aplicaciones: es el caso típico de la multiprogramación.

- Aplicaciones estructuradas: normalmente se implementan con procesos concurrentes (multihebras).
- Estructura del sistema operativo: el propio sistema operativo puede tener aplicaciones estructuradas que trabajen con procesos concurrentes.
- Otra característica que puede presentar un grupo de procesos concurrentes es un evento que se denomina condición de carrera que sucede cuando múltiples procesos o hilos leen y escriben datos de manera que el resultado final depende del orden de ejecución de las instrucciones en los múltiples procesos.

Interacción entre procesos. Sección crítica

Se pueden clasificar las formas en que los procesos interaccionan en base al grado en que perciben la existencia unos de otros, y entonces se tendrían:

- Procesos que no se perciben entre sí, como ocurre con los procesos independientes. En este caso el sistema operativo necesita ocuparse de la competencia por los recursos (disco, archivo, impresora, etc.) regulando el acceso a los mismos.
- Procesos que se perciben indirectamente entre sí, como son los procesos que muestran cierta cooperación en la compartición de un objeto común como por ejemplo un buffer de E/S.
- Procesos que se perciben directamente entre sí, comunicándose entre sí vía el ID del proceso y diseñados para trabajar conjuntamente en cierta actividad.

Estas formas de percepción dan lugar ciertas situaciones que el sistema operativo tendrá que tratar y son:

- La competencia entre procesos por recursos.
- La cooperación entre procesos vía compartición.
- La cooperación entre procesos vía comunicación.

En el caso de procesos en competencia, el sistema operativo deberá afrontar los siguientes problemas: la exclusión mutua, el interbloqueo y la inanición.

La exclusión mutua consiste en excluir a los demás procesos de acceder a un recurso compartido (variable, archivo, etc.) mientras un proceso está accediendo al mismo en modo escritura.

La aplicación de la exclusión mutua puede provocar que las necesidades de recursos de algunos procesos entren en conflicto entre sí, logrando que los procesos se bloqueen indefinidamente, situación que recibe el nombre de interbloqueo o *deadlock*.

La inanición ocurrirá porque a un proceso de baja prioridad podría denegársele indefinidamente el acceso al recurso solicitado (procesador, variable, archivo, impresora, etc.), aunque no suceda un interbloqueo.

Al recurso solicitado se lo llama recurso crítico y a la porción del programa que lo utiliza como la sección crítica del programa. Luego, el problema de la sección crítica consiste en diseñar un protocolo que los procesos puedan usar para cooperar. Cada proceso debe solicitar permiso para entrar en su sección crítica; la sección de código que implementa esta solicitud es la sección de entrada. La sección crítica puede ir seguida de una sección de salida. El código restante se encuentra en la sección restante.

Asimismo, cualquier protocolo diseñado para proteger a la sección crítica deberá cumplir con una serie de requisitos, a saber:

- Exclusión mutua: si el proceso P se está ejecutando en su sección crítica, entonces ningún otro proceso puede ejecutarse en su sección crítica.
- Progreso: si ningún proceso se está ejecutando en su sección crítica, la decisión de que proceso entra en su sección crítica se hará sobre los procesos que desean entrar. Esta decisión debe realizarse en un tiempo finito.
- Espera limitada: debe haber un límite en el número de veces que se permite que los demás procesos entren a ejecutar el código de su sección crítica después de que un proceso haya efectuado una solicitud de entrada y antes de que se conceda la suya.

Las condiciones de competencia se presentan cuando dos o más procesos o hebras intentan acceder simultáneamente a un mismo recurso. Este hecho es muy difícil de detectar ya que el error se produce muy de vez en cuando. Para evitar esta condición, los procesos concurrentes deben ser sincronizados, es decir, utilizar operaciones atómicas para asegurar que los procesos estén libres de interferencias.

Una operación atómica consiste en un conjunto de instrucciones que realiza una sola función lógica durante la cual no se interrumpe al procesador. Esta solución, implementada por hardware en sistemas monoprocesadores, o con *kernels* no apropiativos, consiste en desactivar las interrupciones mientras se ejecuta una operación atómica. En cambio, en sistemas de multiprocesadores, donde resultaría muy

inconveniente aplicar esta solución por el tiempo que llevaría notificar a cada uno de los procesadores, es habitual que se tengan instrucciones especiales que permitan considerar a la sección crítica como una operación atómica, es decir, como una unidad de trabajo no interrumpible. De esta manera es posible resolver el problema de la sección crítica de una manera simple al asegurar que éstas se ejecuten atómicamente.

Comunicación entre procesos

Cuando deba existir una cooperación entre procesos será necesario la implementación de mecanismos de comunicación interprocesos (IPC: *Interprocess Communication*) que les permitan a éstos intercambiar datos e información. Estos mecanismos pueden ser reducirlos a un par de modelos: memoria compartida y paso de mensajes.

En el primer modelo, se establece una región de la memoria para que sea compartida por los procesos cooperativos. De este modo, los procesos pueden intercambiar información leyendo y escribiendo datos en la zona de memoria compartida. En el segundo modelo, la comunicación tiene lugar mediante el intercambio de mensajes entre los procesos cooperativos.

El modelo de paso de mensajes proporcionará al menos dos operaciones, una para el envío del mensaje (*send*) y otra para la recepción del mensaje (*receive*). La forma de implementar este modelo es:

- Con comunicación directa o indirecta.
- Con comunicación síncrona o asíncrona.

Los detalles de cada forma serían los siguientes:

Comunicación directa

Es unidireccional o bidireccional.

La comunicación puede ser sin o con el nombre del proceso.

Cuando es *sin* el nombre del proceso, las operaciones tienen el siguiente diseño:

- *Send* (mensaje) — Envía un mensaje.
- *Receive* (mensaje) — Recibe un mensaje.

Mientras que en el otro caso la comunicación podría ser simétrica o asimétrica. Si la comunicación es simétrica, tanto el proceso transmisor como el proceso receptor deben

nombrar al otro para comunicarse y, entonces, las operaciones tendrán el siguiente diseño:

- *Send* (A, mensaje) — Envía un mensaje al proceso A.
- *Receive* (B, mensaje) — Recibe un mensaje del proceso B.

Pero si la comunicación es asimétrica, sólo el transmisor nombra al receptor, mientras que el receptor no tiene que nombrar al transmisor; siendo el diseño de las operaciones el siguiente:

- *Send* (P, mensaje) — Envía un mensaje al proceso P.
- *Receive* (mensaje) — Recibe un mensaje de cualquier proceso.

Comunicación indirecta a través de buzones o puertos

Un buzón de correo puede considerarse como un objeto abstracto en el que los procesos pueden colocar mensajes y del que pueden eliminar mensajes. Cada buzón tiene un nombre y los procesos pueden comunicarse si comparten un buzón.

Las propiedades de la comunicación indirecta serían las siguientes:

- Un par de procesos se comunican si tienen un buzón compartido.
- Un enlace puede estar asociado a más de dos procesos.
- Los enlaces pueden ser unidireccionales o bidireccionales.
- Entre cada par de procesos en comunicación, puede haber una serie de enlaces diferentes, correspondiendo cada enlace a un buzón.

Si ocurre que los procesos A, B y C comparten el buzón X y el proceso A envía un mensaje al buzón X, mientras que los procesos B y V ejecutan un *Receive* de X, ¿cuáles procesos recibirán el mensaje enviado por A? Todo depende de cuál sea el esquema elegido. Las opciones son las siguientes:

- No permitir que el enlace esté asociado a más de dos procesos.
- No permitir que más de un proceso ejecute una operación de recibir a la vez.
- Permitir que el sistema elija arbitrariamente cual es el proceso que recibirá el mensaje (B o C).

Un buzón puede ser propiedad de un proceso o del sistema operativo:

- Si es propiedad del proceso, éste recibe información y al finalizar desaparece el buzón.

- Si es propiedad del sistema permanece siempre con el sistema.

La capacidad de un buzón puede ser: nula, limitada o ilimitada. En el primer caso el emisor debe bloquearse hasta que el receptor reciba el mensaje. En el segundo, si el buzón está lleno, el emisor debe bloquearse hasta que haya espacio disponible en el mismo, caso contrario, si el buzón no está lleno cuando se envía un mensaje, el mensaje se introduce en el buzón, y el emisor puede continuar con su ejecución. En la última opción, como la capacidad del buzón es potencialmente infinita, y por lo tanto, puede haber cualquier cantidad de mensajes esperando en él, el emisor nunca se bloquea.

Comunicación síncrona o asíncrona

Esta forma de comunicación, también conocida como con bloqueo o sin bloqueo, presenta las opciones siguientes:

- Envío con bloqueo: el proceso emisor se bloquea hasta que el proceso receptor o el buzón reciben el mensaje.
- Envío sin bloqueo: el proceso transmisor envía el mensaje y continúa operando.
- Recepción con bloqueo: el proceso receptor se bloquea hasta que hay un mensaje disponible.
- Recepción sin bloqueo: el proceso receptor extrae un mensaje válido o un mensaje nulo.

Posibles soluciones al tema de la sección crítica

Inhabilitación de interrupciones

Esta solución tiene las características siguientes:

- Es útil dentro del sistema operativo, pero no apropiada como mecanismo de exclusión mutua para los procesos de usuario.
- Sólo funciona en arquitecturas monoprocesadoras.
- La sincronización debe efectuarse en el código del programa y si este entra en un bucle infinito el sistema se cuelga completamente.
- No es aplicable en sistemas de tiempo real.
- Tiene espera activa, es decir que el proceso no suspende su ejecución, sino que entra en un bucle en el que consulta de forma continua un valor que le permite

continuar su ejecución o no. De esta manera, se desperdician ciclos del procesador en procesos que no se pueden ejecutar.

Sincronización mediante instrucciones de máquina especiales

El uso de instrucciones de máquina especiales para conseguir la exclusión mutua tiene sus ventajas y desventajas. Las *ventajas* serían las siguientes:

- Se puede aplicar a cualquier número de procesos tanto en un sistema monoprocesador como en uno multiprocesador de memoria principal compartida.
- Es simple y fácil de controlar.
- Se lo puede utilizar para dar soporte a múltiples secciones críticas.

Mientras que sus *desventajas* serían:

- Emplea espera activa.
- Puede generar inanición.
- Puede producirse un interbloqueo. Por ejemplo, en un sistema monoprocesador con un sistema operativo apropiativo que trabaja con prioridades, si el proceso A ejecuta la instrucción especial y entra en su sección crítica y entonces A es interrumpido para darle el procesador al proceso B, que tiene mayor prioridad, y ahora B intenta utilizar el mismo recurso que A, se le denegará el acceso, dado el mecanismo de exclusión mutua. Así que caerá en un bucle de espera activa, mientras que A no será seleccionado para continuar porque tiene menor prioridad que B.

Algoritmo de Dekker

Resuelve el problema de la exclusión mutua, pero mediante un programa complejo y muy difícil de seguir y de demostrar su corrección.

Algoritmo de Peterson

La solución de Peterson se restringe a dos procesos que van alternando la ejecución de sus secciones críticas y de sus secciones restantes. Consiste en una variable global señal (Vector) que indica la posición del proceso respecto de la exclusión mutua y una variable global turno que resuelve los conflictos de simultaneidad.

Semáforos

Es una herramienta de software que se utiliza para resolver los problemas de acceso a la sección crítica y los problemas de sincronización. Un semáforo S es una variable entera que actúa como señal para coordinar un conjunto de procesos secuenciales cooperantes, a la que sólo se puede acceder a través de 3 operaciones atómicas: $\text{wait}(S)$, $\text{signal}(S)$ e $\text{init}(S, \text{valor inicial})$ y que puede no requerir espera activa.

Para transmitir una señal vía el semáforo S , el proceso ejecutará la primitiva $\text{signal}(S)$. Para recibir una señal vía el semáforo S , el proceso ejecutará la primitiva $\text{wait}(S)$; si la correspondiente señal no se ha transmitido todavía, el proceso se suspenderá hasta que la transmisión tenga lugar.

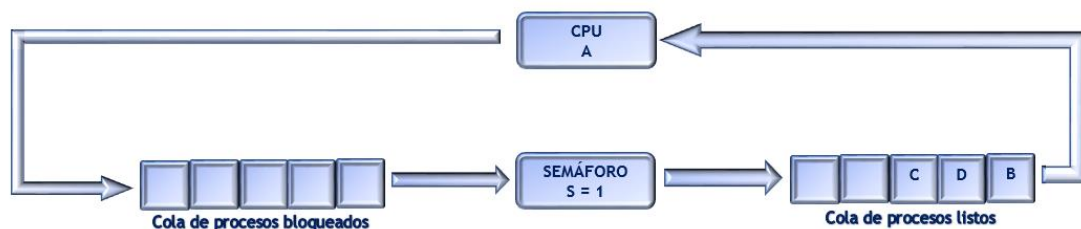
Existen dos tipos de semáforo:

- Semáforo contador o general que puede ser inicializado con cualquier valor entero positivo y luego puede tomar cualquier valor, inclusive negativo.
- Semáforo binario o mutex, cuyo valor entero sólo puede ser 0 ó 1 y es simple de implementar.

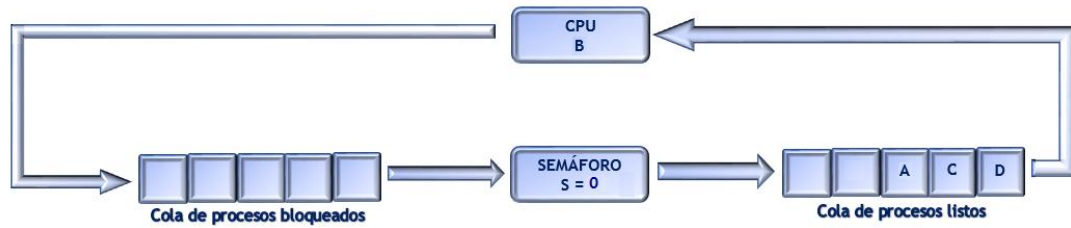
Cabe agregar que también se puede implementar un semáforo contador como un semáforo binario.

En el siguiente ejemplo de semáforos contadores, los procesos A, B y C dependen del proceso D. Luego se tiene que:

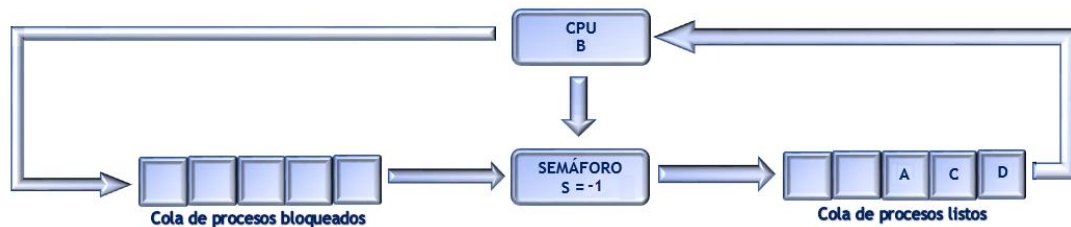
- A está en ejecución, B, D y C están en la cola de listos y el contador del semáforo (S) vale 1 (esto dice que hay un resultado de D disponible):



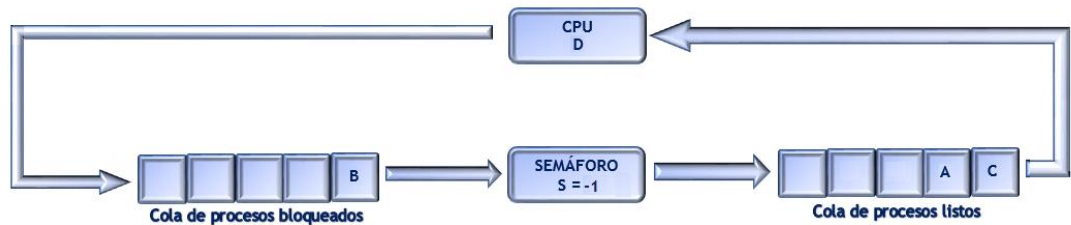
- Cuando A ejecuta un wait(S) el contador del semáforo queda en 0 y A puede continuar ejecutando. Al terminar A, pasa a la cola de listos y comienza a ejecutar B:



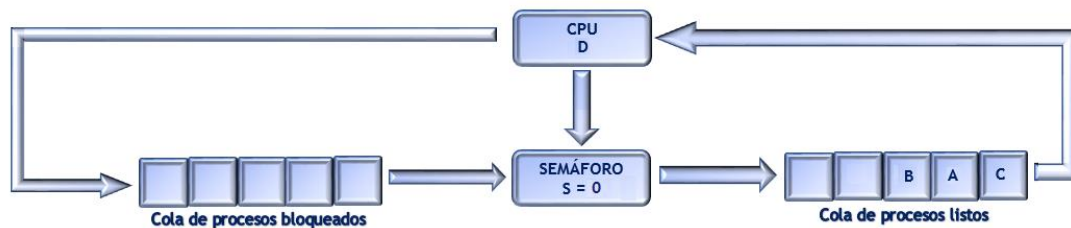
- Cuando B necesite tomar valores de D, ejecutará un wait(S) lo que pone el contador del semáforo en -1 y B se suspende:



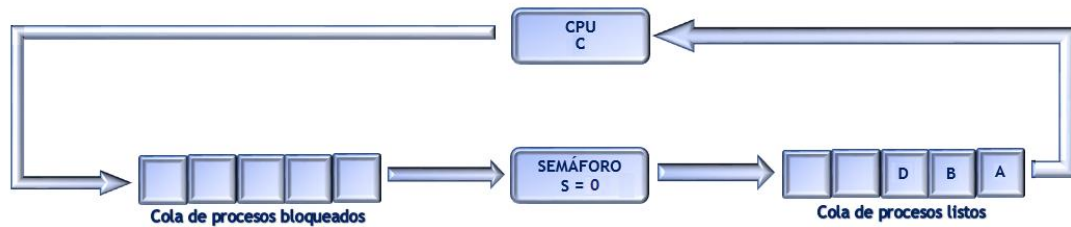
- Ahora ejecuta D:



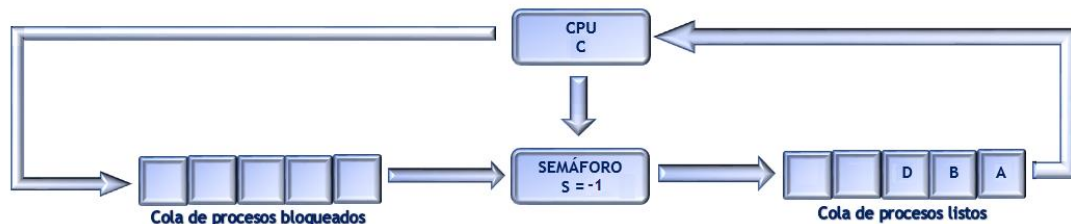
- Cuando completa un resultado ejecuta un signal(S) que coloca al contador del semáforo en 0 y permite a B volver a pasar a la cola de listos:



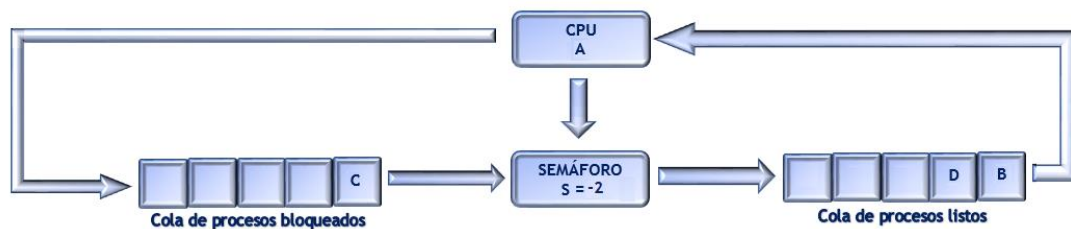
- Al terminar D se lo devuelve a la cola de listos y comienza a ejecutarse C:



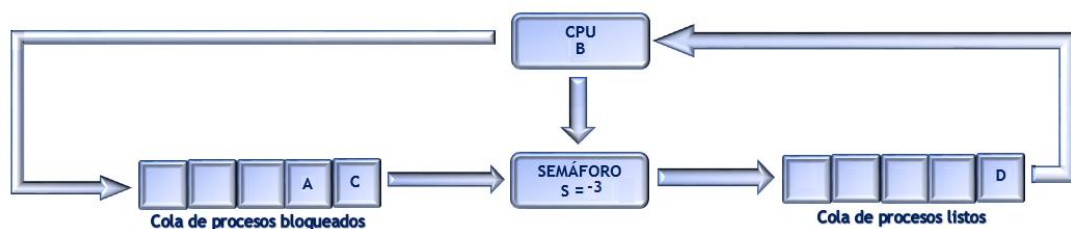
- Cuando C ejecuta un wait(S) el semáforo pasa a -1 y se suspende C pasando a la cola de suspendidos:



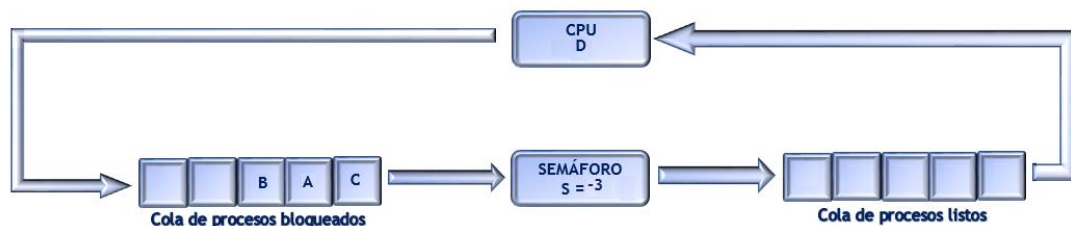
- Comienza a ejecutarse A y cuando ejecuta un wait(S) el semáforo pasa a -2:



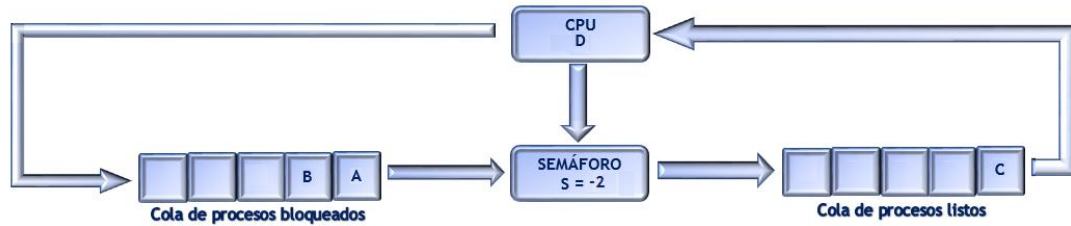
- Entonces se suspende A que pasa a la cola de suspendidos y comienza a ejecutarse B. Cuando B ejecuta un wait(S) el semáforo pasa a -3:



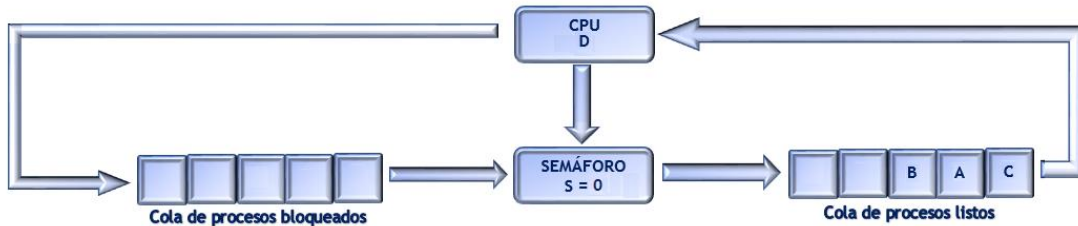
- Luego se suspende B, pasa a la cola de suspendidos y D reanuda su ejecución:



- Al finalizar D hace un $signal(S)$ lo que lleva al semáforo a -2 y así permite a C pasar a la cola de listos:

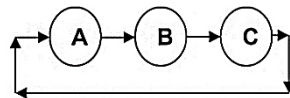


- En los ciclos posteriores de D el semáforo pasa a 0 y se liberan los procesos A y B:



Primer ejemplo del uso de semáforos mutex

Supóngase que se desea sincronizar a los procesos A, B y C con el objetivo de lograr la secuencia de ejecución ABCABC... representada en el siguiente grafo:



Para lograr esto se deberán indicar los semáforos binarios necesarios, sus valores iniciales y las operaciones *wait* y *signal* necesarias mediante el uso de los 2 cuadros siguientes. En el primero se indicarán las operaciones *wait* y *signal* que permiten obtener la secuencia deseada.

PROCESOS		

Mientras que el segundo se usará para establecer el valor inicial de cada semáforo y su actualización en las distintas instancias de tiempo:

		Semáforos		
	Valor inicial			
Instancia	Proceso			

0				
1				
2				
3				
4				
5				
:	:	:	:	:

Por lo tanto, en el primer cuadro se vería la sucesión lógica de los semáforos que permiten la secuencia de sincronización pedida:

PROCESOS		
A	B	C
:	:	:
Wait (SA)	Wait (SB)	Wait (SC)
Sección crítica de A	Sección crítica de B	Sección crítica de C
Signal (SB)	Signal (SC)	Signal (SA)
:	:	:

Mientras que en el cuadro siguiente se verá el valor inicial de cada semáforo y su actualización en las distintas instancias de tiempo:

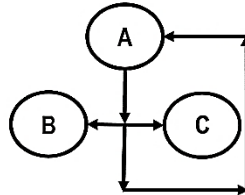
		Semáforos		
		SA	SB	SC
	Valor inicial	1	0	0
Instancia	Proceso			
0	A	0	0	0
1	A	0	1	0
2	B	0	0	0
3	B	0	0	1
4	C	0	0	0
5	C	1	0	0
6	A	0	0	0
7	A	0	1	0

Dos consideraciones a tener en cuenta:

1. En el primer cuadro la cantidad de columnas variará en función de la cantidad de procesos.
2. En segundo cuadro la cantidad de columnas y filas variará en función de la cantidad de semáforos y de instancias consideradas respectivamente.

Segundo ejemplo del uso de semáforos mutex

Se quiere implementar una programación concurrente que responda a la secuencia de ejecución A(BoC)A(BoC)... de los procesos A, B y C, representada por el grafo:



En el siguiente cuadro se muestra la sucesión lógica de los semáforos que permite la secuencia de sincronización pedida:

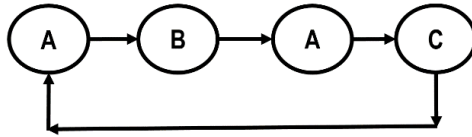
PROCESOS		
A	B	C
:	:	:
Wait (SA)	Wait (SBC)	Wait (SBC)
Sección crítica de A	Sección crítica de B	Sección crítica de C
Signal (SBC)	Signal (SA)	Signal (SA)
:	:	:

Mientras que en el cuadro siguiente se ve el valor inicial de cada semáforo y su actualización en las distintas instancias de tiempo:

		Semáforos	
		SA	SBC
	Valor inicial	1	0
Instancia	Proceso		
0	A	0	0
1	A	0	1
2	B o C	0	0
3	B o C	1	0
4	A	0	0

Tercer ejemplo del uso de semáforos mutex

Se quiere encontrar la secuencia lógica y los valores iniciales de los semáforos que permitan sincronizar los procesos A, B y C según la secuencia de ejecución ABACABAC... cuyo grafo es:



El primer cuadro muestra la sucesión lógica de los semáforos que permite la secuencia de sincronización pedida:

PROCESOS		
A	B	C
: Wait (SA)	: Wait (SB) Wait (SX)	: Wait (SC) Wait (SX)
Sección crítica de A	Sección crítica de B	Sección crítica de C
Signal (SX) :	Signal (SA) Signal (SC) :	Signal (SA) Signal (SB) :

Mientras que el cuadro siguiente muestra el valor inicial de cada semáforo y su actualización en las distintas instancias de tiempo:

		Semáforos			
		SA	SB	SC	SX
	Valor inicial	1	1	0	0
Instancia	Proceso				
0	A	0	1	0	0
1	A	0	1	0	1
2	B	0	0	0	0
3	B	1	0	1	0
4	A	0	0	1	0
5	A	0	0	1	1
6	C	0	0	0	0
7	C	1	1	0	0

Monitores

El monitor es una construcción de sincronización de alto nivel que permite compartir en forma segura un tipo de dato abstracto entre procesos concurrentes. Existen dos clases: el monitor de Hoare y el monitor de Lampson / Redell que, aunque presentan diferencias, tienen algunas características comunes como son las siguientes:

- Está formado por variables, estructuras de datos y procedimientos.
- Los procesos pueden invocar a los procedimientos del monitor, pero no a sus variables ni a las estructuras de datos.

- Un monitor garantiza la exclusión mutua asegurando que en un momento dado sólo un procedimiento del monitor puede estar activo. Esto lo hace el código generado por el compilador.
- Forman parte del lenguaje de programación.
- El programador debe preocuparse por transformar las secciones críticas en procedimientos del monitor. No necesita saber cómo hace el monitor para implementar la exclusión mutua.

Bibliografía consultada

- Carretero Pérez, J., De Miguel Anasagasti, P., García Carballeira, F. y Pérez Costoya, F. (2001). *Sistemas Operativos: Una visión aplicada* (2ª ed.). Buenos Aires: Mc Graw Hill.
- Silberschatz, A. y Galvin, P. B. (2006). *Fundamentos de Sistemas Operativos* (7ª ed.). México: Addison Wesley Logman.
- Stallings, W. (2005). *Sistemas operativos: Principios de diseño e interioridades* (5ª ed.). Madrid: Pearson/Prentice Hall.
- Tanenbaum, A. S. y Van Steen, M. (2009). *Sistemas operativos modernos* (3ª ed.). México: Pearson/Prentice Hall.